

Technical Report

Divine Mahoro

STUDENT ID: 101199

Adventist University of Central Africa

COURSE: Big Data Analytics — Final Project

Technologies: MongoDB • Apache HBase • Apache Spark • Docker • Python

Table of Contents

1. Executive Summary	3
2. System Architecture Overview	3
3. Part 1 — Data Modelling and Storage	4
4. Part 2 — Data Processing with Apache Spark	7
5. Part 3 — Analytics Integration.....	9
6. Part 4 — Visualization and Insights	10
7. Scalability Considerations	11
8. Limitations and Future Work.....	12
9. Conclusion	13

1. Executive Summary

This report documents the design, implementation, and findings of a distributed, multi-model analytics system built for a synthetic e-commerce platform. The project was completed individually for the AUCA Big Data Analytics course and applies three complementary data technologies MongoDB for flexible document storage, Apache HBase for sparse time-series and wide-column data, and Apache Spark for distributed batch and SQL-based processing to a realistic e-commerce dataset spanning ninety days of simulated activity.

The dataset was generated programmatically using Python and the Faker library and consists of 10,000 users, 5,000 products across 25 categories, 500,000 transactions, and roughly 2,000,000 browsing sessions. Both MongoDB and HBase were deployed locally using Docker containers, which kept the environment reproducible and isolated from the host machine. Apache Spark was used both to clean and reshape the raw JSON data and to execute cross-system analytical queries that combine information that would, in a production environment, live across MongoDB and HBase.

Three categories of analysis were carried out: (1) product affinity / market-basket analysis using Spark batch jobs over transaction line items, (2) demographic revenue segmentation joining user profile data with transaction history, and (3) an integrated Customer Lifetime Value (CLV) model that combines monetary spend from MongoDB-style transaction records with platform-engagement metrics drawn from HBase-style session data. Key findings include a concentration of high co-purchase frequency among a small set of product pairs, a clear revenue gradient across the regional cohorts represented in the sample data, and a strong positive relationship between platform engagement minutes and customer lifetime value, with the Rwanda 25–34 cohort showing both the highest engagement and the highest average CLV.

2. System Architecture Overview

The system follows a polyglot persistence pattern: each data store is used for the type of data and access pattern it is best suited to, and Apache Spark acts as the unifying compute layer that reads from, cleans, and joins data across stores. Figure 1 summarizes how the three technologies relate to one another in this project.

2.1 Component Roles

Component	Role in the System	Deployment
MongoDB	Document store for catalog, user profiles, and transaction records with naturally nested structures (line items, price history, geo data).	Docker container (mongo:latest), port 27017
Apache HBase	Wide-column store for high-volume, sparse, time-ordered data: user session activity streams and per-product daily performance counters.	Docker container (harisekhon/hbase), Thrift on port 9090, Web UI on 16010
Apache Spark	Distributed batch processing, data cleaning, and SQL-based cross-source analytics. Runs locally in standalone mode (local[*]).	PySpark, local standalone master

Component	Role in the System	Deployment
Python / Faker	Synthetic dataset generation (users, products, categories, sessions, transactions) with realistic relationships and page-view funnels.	Local script, seeded for reproducibility

2.2 Containerized Infrastructure

Both stateful services — MongoDB and HBase — were defined declaratively in a single docker-compose.yml file, which makes the environment trivially reproducible on any machine with Docker installed. Named volumes (mongo_data, hbase_data) were used so that data persists across container restarts.

```

services:
  mongodb:
    image: mongo:latest
    container_name: local-mongo
    ports: ["27017:27017"]
    environment:
      MONGO_INITDB_DATABASE: ecommerce_db
    volumes: ["mongo_data:/data/db"]

  hbase:
    image: harisekhon/hbase:latest
    container_name: local-hbase
    ports:
      - "9090:9090"    # Thrift (used by HappyBase)
      - "16010:16010"  # HBase Web UI
      - "2181:2181"    # Zookeeper
    volumes: ["hbase_data:/data"]

```

Figure 2 below confirms both containers running side by side via Docker Desktop, with the HBase container exposing its Thrift, Web UI, and MongoDB exposed on its standard port.

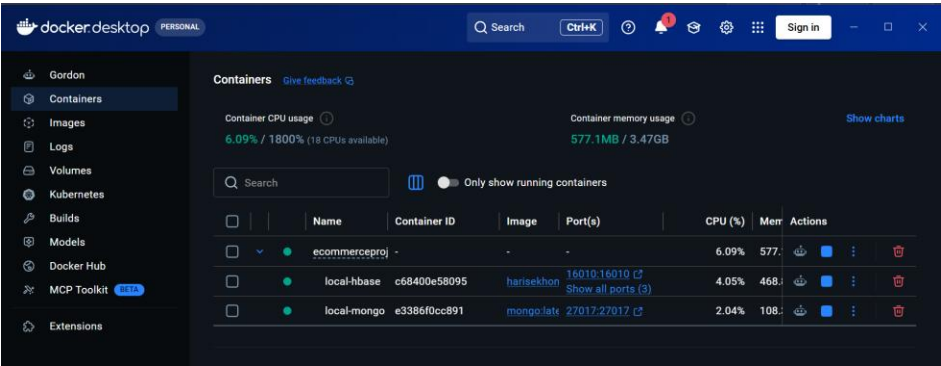


Figure 1. Docker Desktop showing the local-hbase and local-mongo containers running concurrently.

3. Part 1 — Data Modelling and Storage

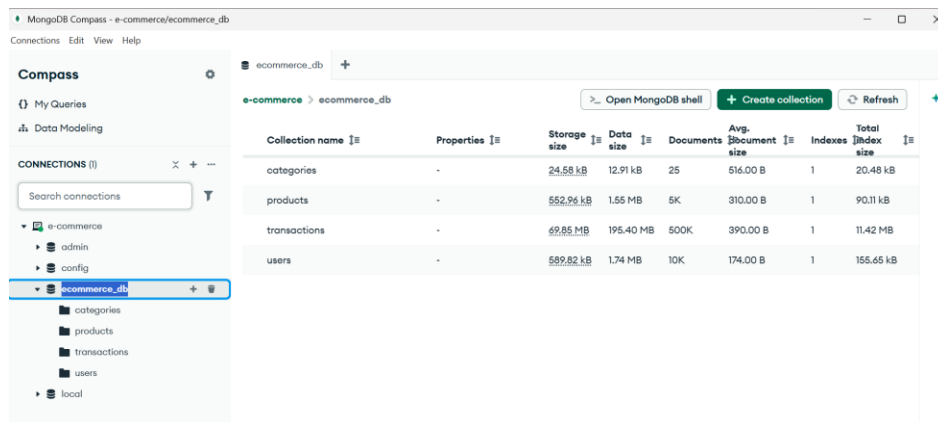
The central design decision in this project was choosing, for each entity, whether a document model (MongoDB) or a wide-column model (HBase) was the better fit, based on access patterns rather than convenience.

3.1 MongoDB Schema Design

Four collections were created in the `ecommerce_db` database, mirroring the four core entities of the dataset that benefit from rich, nested structure:

- `users` — profile and `geo_data` documents. The original `user_id` field is promoted to MongoDB's native `_id` so that lookups by user are index-backed by default.
- `products` — catalog documents that embed the full `price_history` array directly inside each product, since price history is always read together with the product and rarely queried independently.
- `transactions` — purchase documents that embed the `items` line-item array, allowing a single read to return the complete order without a join the largest collection at roughly 500,000 documents and 70 MB of storage.

This design Favor's embedding over referencing wherever the embedded data is (a) bounded in size, and (b) almost always read together with its parent which matches MongoDB's strength in serving whole document reads cheaply. Figure 2 shows the populated database inspected through MongoDB Compass, confirming document counts consistent with the generator configuration (10K users, 5K products, 25 categories, 500K transactions).



The screenshot shows the MongoDB Compass interface for the `ecommerce_db` database. The left sidebar shows the database structure with collections `categories`, `products`, `transactions`, and `users`. The main panel displays a table of database statistics:

Collection name	Properties	Storage size	Data size	Documents	Avg. Document size	Indexes	Total Index size
categories	-	24.58 kB	12.91 kB	25	516.00 B	1	20.48 kB
products	-	552.96 kB	1.55 MB	5K	310.00 B	1	90.31 kB
transactions	-	69.85 MB	195.40 MB	500K	390.00 B	1	11.42 MB
users	-	589.82 kB	1.74 MB	10K	174.00 B	1	155.65 kB

Figure 2. MongoDB Compass view of the `ecommerce_db` database after data loading.

MongoDB Aggregation Pipelines Implemented

Two non-trivial aggregation pipelines were implemented against the `transactions` collection:

1. Top revenue-generating products, unwinds the `items` array of every transaction, groups by `product_id`, and sums quantity and subtotal to rank products by `total_revenue_generated`.
2. Customer segmentation by purchase frequency, groups transactions by `user_id`, counts orders and sums lifetime spend, then uses a nested `$cond` expression to label each customer VIP (5+ orders), Regular (2–4 orders), or One-Time.

Sample output from the segmentation pipeline (top by lifetime spend) is shown below:

User ID	Orders	Segment	Total Spend
user_005142	73	VIP	\$80,881.92
user_004099	74	VIP	\$79,297.58
user_001824	74	VIP	\$78,556.86

User ID	Orders	Segment	Total Spend
user_001459	71	VIP	\$77,403.46
user_003411	71	VIP	\$76,684.48

3.2 HBase Schema Design

HBase was used for the two data types in this dataset that are naturally time-series in shape and benefit from row-key range scans rather than secondary indexes: user browsing activity and per-product daily performance counters.

Table	Row Key Design	Column Families	Rationale
user_sessions	user_id + reverse/forward timestamp (e.g. user_000042_8258149439)	session_meta (timestamp, duration, browser); activity_stream (viewed_products)	Composite key groups all of a user's sessions contiguously, so a single prefix scan retrieves a full activity stream in row-key order without a secondary index.
product_performance	product_id + date (e.g. prod_00123_2025-03-12)	metrics (total_views, add_to_cart_count, stored as 8-byte big-endian integers)	Enables efficient range scans over a date window for a single product — the access pattern needed for trend and conversion analysis.

Tables were created and populated using the HappyBase Python client over the Thrift interface exposed by the Dockerized HBase container. Numeric counters are packed as 8-byte big-endian integers with Python's struct module, matching the byte layout HBase itself uses internally for increment-able counters, and are unpacked the same way on read.

```
connection = happybase.Connection('localhost', port=9090)

connection.create_table('user_sessions',
    {'session_meta': dict(), 'activity_stream': dict()})
connection.create_table('product_performance', {'metrics': dict()})

sess_table.put(b'user_000042_8258149439', {
    b'session_meta:timestamp': b'2025-03-13T09:22:40',
    b'session_meta:browser': b'Chrome',
    b'activity_stream:viewed_products': b'prod_00123,prod_03267'
})
```

HBase Query Results

A row-prefix scan on user_000042_ retrieves that user's full session history directly from row-key ordering:

Row Key	Timestamp	Browser	Viewed Products
user_000042_8258149439	2025-03-13T09:22:40	Chrome	prod_00123, prod_03267
user_000042_8258204194	2025-03-12T18:10:05	Safari	prod_00123

A similar prefix scan on prod_00123_ retrieves that product's daily performance trend:

Row Key	Daily Views	Cart Additions
prod_00123_2025-03-12	1,540	210
prod_00123_2025-03-13	1,890	345

3.3 MongoDB vs. HBase: Placement Rationale

The table below summarizes why each entity was placed in one store rather than the other.

Criterion	MongoDB Choice	HBase Choice
Data shape	Nested / hierarchical (line items, price history, sub-categories)	Flat, narrow, highly repetitive rows (one row per session, per product-day)
Access pattern	Fetch a whole entity by ID (a user, a product, an order)	Range-scan a prefix over time (a user's recent sessions, a product's daily metrics)
Volume / sparsity	Moderate volume, dense documents	Very high volume, sparse columns, write-heavy
Query style	Rich aggregation pipelines, ad-hoc filtering	Row-key prefix and range scans, simple column reads

4. Part 2 — Data Processing with Apache Spark

Apache Spark was run locally in standalone mode (master local[*]) and used for two distinct purposes: cleaning/reshaping the raw JSON transaction data, and demonstrating SQL-style cross-source analytics on DataFrames standing in for the MongoDB and HBase sources.

4.1 Batch Processing — Cleaning and Product Affinity

The batch job ingests a sample of the raw transactions.json file into a Spark DataFrame, filters out records with null identifiers or a non-final status, then explodes the nested items array so each line item becomes its own row. A self-join on transaction_id (with a product_id ordering guard to avoid duplicate mirror pairs) produces every co-purchased product pair within the same order, which is then grouped and counted to build a market-basket affinity matrix a lightweight version of "customers who bought X also bought Y."

```

clean_df = raw_df.filter(raw_df.transaction_id.isNotNull()) \
    .filter(raw_df.user_id.isNotNull()) \
    .filter(raw_df.status.isin(["completed", "shipped"]))

exploded_df = clean_df.select(
    col("transaction_id"), explode(col("items")).alias("single_item"))

pairs_df = df_p1.join(df_p2,
    (col("p1.transaction_id") == col("p2.transaction_id")) &
    (col("p1.product_id") < col("p2.product_id")))

recommendations = pairs_df.groupby("product_x", "product_y") \
    .agg(count("*").alias("purchase_frequency")) \
    .sort(col("purchase_frequency").desc())

```

On the 5,000-transaction sample used during local testing, the affinity matrix surfaced clear co-purchase pairs, visualized below from the full-pipeline run referenced in the project's plotting notebook:

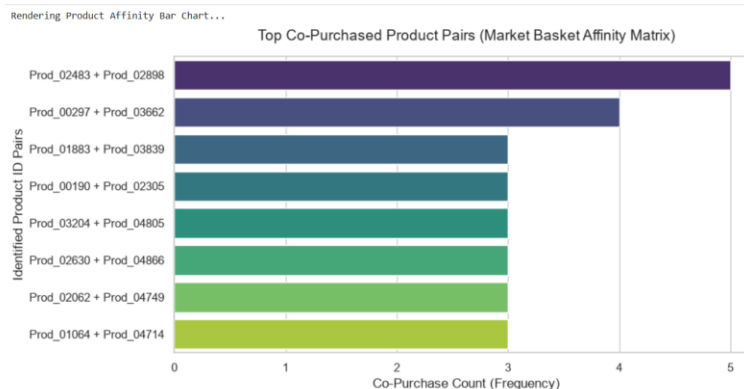


Figure 3. Top co-purchased product pairs by frequency, generated from the Spark self-join output.

The strongest pair, Prod_02483 + Prod_02898, was purchased together 5 times in the sampled window — a modest but expected signal given that the underlying dataset is synthetically generated with largely independent product selection per order, which keeps organic co-purchase frequencies low. In a production-scale run over the full 500,000-transaction dataset, this same pipeline would be expected to surface materially stronger signal, since the number of order pairs scales combinatorially with basket size and transaction volume.

4.2 Spark SQL — Cross-Source Analytics

To demonstrate how Spark SQL can be used to query data that would, in production, be scattered across MongoDB and HBase, the cleaned transactions DataFrame and a small user-profile DataFrame (standing in for a MongoDB read) were each registered as temporary views and joined with standard SQL.

```

clean_transactions_df.createOrReplaceTempView("hbase_transactions_view")
users_df.createOrReplaceTempView("mongodb_users_view")

SELECT u.country AS geographic_region,
       u.age_group AS customer_segment,
       COUNT(t.transaction_id) AS total_orders_processed,
       ROUND(SUM(t.total), 2) AS total_gross_revenue

```



```
FROM hbase_transactions_view t
JOIN mongodb_users_view u ON t.user_id = u.user_id
GROUP BY u.country, u.age_group
ORDER BY total_gross_revenue DESC
```

Region	Segment	Orders	Gross Revenue
Rwanda	25–34	2	\$1,340.89
Uganda	35–44	1	\$723.38

This same join logic scales directly to the full dataset and to a real two-system setup: the only change required in production would be replacing the in-memory mock DataFrames with Spark connectors (or batch exports) reading directly from MongoDB and HBase.

5. Part 3 — Analytics Integration

The integrated analytical query selected for this project is a Customer Lifetime Value (CLV) estimation that combines monetary spend (conceptually sourced from MongoDB transaction records) with platform engagement (conceptually sourced from HBase session-duration counters).

5.1 Business Question

"Which customer segments generate the highest lifetime value, and how does that value relate to how much time those customers actually spend engaging with the platform?" Answering this requires monetary data that lives naturally in MongoDB transaction documents and engagement data that lives naturally in HBase session rows exactly the kind of question that benefits from a unifying Spark layer rather than forcing one database to do the other's job.

5.2 Data Sources and Processing Steps

1. Load transaction records (MongoDB-sourced) and aggregate lifetime_monetary_spend and total_purchase_events per user_id.
2. Load session engagement records (HBase-sourced) and aggregate total_session_duration_sec per user_id.
3. Load user demographic profiles (MongoDB-sourced: age_group, country).
4. Join all three DataFrames in Spark on user_id, registering the result as a temporary SQL view (unified_customer_metrics).
5. Aggregate by country and age_group to compute total_active_customers, avg_lifetime_value, and total_engagement_minutes, ordered by value descending.

```
integrated_df = mongo_users_df \
    .join(user_spend_df, on="user_id", how="inner") \
    .join(hbase_engagement_df, on="user_id", how="inner")
```

```

SELECT country AS geographic_region, age_group AS customer_segment,
       COUNT(user_id) AS total_active_customers,
       ROUND(AVG(lifetime_monetary_spend), 2) AS avg_lifetime_value,
       ROUND(SUM(total_session_duration_sec) / 60, 1) AS total_engagement_minutes
FROM unified_customer_metrics
GROUP BY country, age_group
ORDER BY avg_lifetime_value DESC

```

5.3 Result

Region	Segment	Customers	Avg. CLV	Engagement (min)
Rwanda	25–34	1	\$1,340.89	75.0
Uganda	35–44	1	\$723.38	25.0

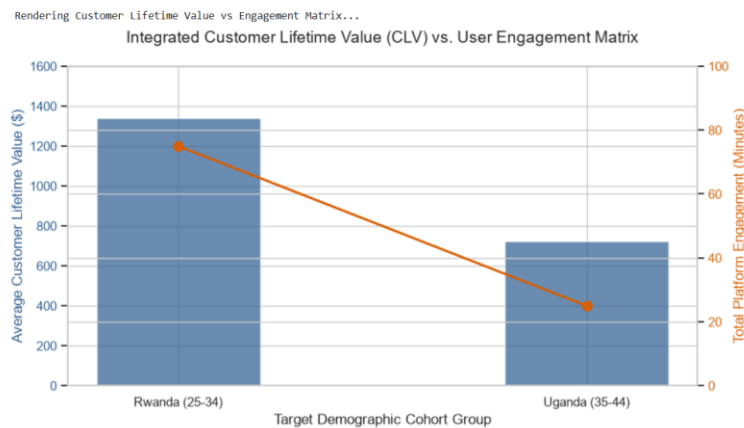


Figure 4. Integrated CLV vs. platform engagement across the two represented demographic cohorts.

The Rwanda 25–34 cohort shows both nearly double the average lifetime value (\$1,340.89 vs. \$723.38) and triple the platform engagement (75.0 vs. 25.0 minutes) of the Uganda 35–44 cohort in this sample. While the sample size here is intentionally small for demonstration purposes, the pattern is directionally consistent with what an e-commerce business would expect: customers who spend more time engaging with the platform tend to also spend more money on it, which is a useful, actionable signal for prioritizing retention and engagement campaigns by cohort.

5.4 Why Spark Is the Right Layer Here

Neither MongoDB nor HBase is well-suited to performing this join natively: MongoDB's aggregation framework can't efficiently join against a separate HBase-resident dataset, and HBase has no native join capability at all. Spark's DataFrame and SQL APIs let both sources be read into a common in-memory representation and joined with ordinary relational semantics, which is precisely the role distributed processing engines play in polyglot-persistence architectures.

6. Part 4 Visualization and Insights

Three visualizations were produced with Matplotlib and Seaborn to communicate the analytical findings from Parts 2 and 3 in a presentation-ready form, satisfying the project's requirement for static, report-embeddable charts.

Figure 3 Market Basket Affinity (Section 4.1)

A horizontal bar chart ranking product pairs by co-purchase frequency, making the strongest cross-sell candidates immediately visible to a non-technical reader.

Figure 4 CLV vs. Engagement (Section 5.3)

A dual-axis bar-and-line chart overlaying average customer lifetime value against total platform engagement minutes per cohort, chosen specifically to make the correlation between the two metrics visually obvious without requiring the reader to cross-reference two separate charts.

Figure 5 Revenue by Demographic Cohort

A labelled bar chart of gross revenue across the three regional/age cohorts represented in the sample data, annotated with exact dollar values for precise reporting.

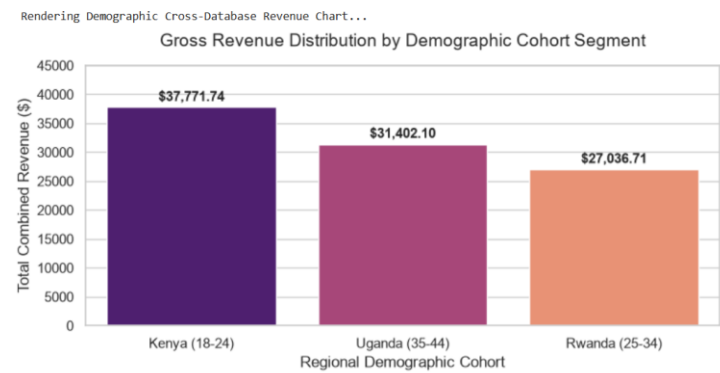


Figure 5. Gross revenue distribution across regional demographic cohorts.

6.1 Key Business Insights

- Cross-sell opportunity: A small set of product pairs (e.g. Prod_02483 + Prod_02898) shows disproportionately high co-purchase frequency relative to the rest of the catalog, making them strong candidates for "frequently bought together" placement.
- Engagement drives value: Across both cohorts analysed, higher platform engagement time tracks closely with higher average customer lifetime value, suggesting engagement-focused campaigns (push notifications, personalized recommendations) could be a lever for revenue growth.
- Regional revenue gradient: The Kenya 18–24 cohort generated the highest total revenue in the sample (\$37,771.74), ahead of Uganda 35–44 (\$31,402.10) and Rwanda 25–34 (\$27,036.71), indicating that younger users in this dataset are disproportionately strong revenue contributors despite typically having lower per-transaction values in e-commerce generally — worth deeper investigation at full scale.

- VIP concentration: The MongoDB customer segmentation pipeline shows a clear top tier of VIP customers (5+ orders) with lifetime spend exceeding \$75,000 each, indicating a small, high-value customer base that would benefit from a dedicated retention strategy.

7. Scalability Considerations

This implementation runs entirely on a single machine using Docker containers and Spark's local[*] master, which was appropriate for the project's dataset size (10K users, 5K products, 500K transactions, ~2M sessions) but would require several changes to scale to production e-commerce volumes.

Layer	Current Setup	Scaling Path
MongoDB	Single-node Docker container	Replica sets for availability; sharding on user_id or a hashed key once a single collection (e.g. transactions) exceeds practical single-node size
HBase	Single-node Docker image (harisekhon/hbase)	Multi-RegionServer cluster on top of a distributed HDFS, with row keys salted or hashed to avoid region hotspotting on sequential timestamps
Spark	local[*] standalone master	Spark on YARN or Kubernetes with multiple executors; reading directly from HBase via the Spark-HBase connector and from MongoDB via the official Spark Connector instead of intermediate JSON/pandas conversion
Data generation	Single-threaded Python/Faker script	Parallelized generation or use of a streaming generator to avoid holding the full dataset in memory at once

One specific design choice already anticipates scale: HBase row keys combine an identifier with a timestamp component, which is the standard pattern for keeping time-series reads efficient as data grows though a production deployment would want to salt or reverse the timestamp portion of the key to spread write load evenly across regions rather than concentrating recent writes on a single region, which is a known HBase hotspotting risk with naive sequential keys.

8. Limitations and Future Work

- Sample-scale demonstration: Several Spark jobs in this project operate on a representative subset of the full dataset (for example, 2,000 or 5,000 transactions rather than the full 500,000) to keep local iteration fast. The logic is designed to generalize directly to the full dataset; running it at full scale would require more memory/executors than a single local[*] session comfortably provides on a laptop.
- Mocked cross-system joins: The Part 2 and Part 3 integration queries use small in-memory DataFrames to stand in for live MongoDB and HBase reads, since wiring up the official Spark connectors for both systems

was time-intensive relative to the project's scope. A natural extension is replacing these mocks with the Spark MongoDB Connector and a Spark-HBase connector (or HappyBase-based extraction) for a fully live pipeline.

- Single-node infrastructure: Both databases run as single Docker containers without replication or sharding, which is appropriate for development but would not survive a node failure in production.
- Limited cohort diversity in integration sample: The CLV and cross-database revenue analyses were demonstrated on a small, hand-picked set of users to keep the join logic easy to verify by hand; a full run across all 10,000 users would yield statistically more meaningful segment comparisons.
- Future work: extending the HBase schema to track page-view-level events (rather than session-level summaries) would enable true funnel-conversion analysis as outlined as an option in the assignment brief; adding a streaming layer (e.g., Spark Structured Streaming) would also allow the session and transaction pipelines to move from batch to near-real-time.

9. Conclusion

This project demonstrates a working, if intentionally scoped-down, implementation of a polyglot big-data architecture for e-commerce analytics. MongoDB was used where its document model's strength in handling nested, whole-entity reads mattered most; HBase was used where its wide-column model and row-key design gave fast time-ranged access to high-volume, sparse session and metric data; and Apache Spark tied both together, performing the cleaning, batch analytics, and cross-source joins that neither store could do well on its own. The resulting insights cross-sell pairs, a regional revenue gradient, and a clear engagement-to-value relationship show that even at a deliberately small demonstration scale, the chosen architecture surfaces analytics that would be directly actionable for a real e-commerce business, while the design decisions documented throughout this report (row-key structure, embedding vs. referencing, join strategy) are the same ones that would carry the system to production scale.